Departamento de Computación y Sistemas Inteligentes
Computación y Estructuras Discretas III
Follow-Up 3: Finite State Transducers

Name: _____

Objetivos — Unidad 1: Lenguajes Regulares y Autómatas

Definir formalmente los distintos tipos de autómatas finitos (deterministas, no deterministas, no deterministas con transiciones lambda, transductores), dar ejemplos, identificar sus elementos y conocer sus aplicaciones. Entender y aplicar las nociones de lenguajes y expresiones regulares, como de autómatas de estado finito, para el reconocimiento de patrones, procesamiento, validación y extracción de texto usando un lenguaje de programación.

Context

The word `musica` (music in Spanish and Italian) has several European language translations that use the Latin alphabet: `music` (English), `musique` (French), `musik` (German), `musiek` (Dutch/Afrikaans), and `musikk` (Norwegian).

Point 1 — Finite State Transducer: Multilingual Word Translation **[100%]**

Design a **non-deterministic FST** that reads the string `musica` and produces **exactly** the following five translations as its output set: `music`, `musique`, `musik`, `musiek`, `musikk`. Your FST must not produce any string beyond these five; every accepting path must lead to one of them and no other.

1. **[50%]** Design the FST **by hand on paper**.
   a. Draw the complete state diagram. Label each transition with $u : v$ notation (read u, write v); use λ or ϵ when a transition produces no output symbol. Mark the initial state with an incoming arrow ($\rightarrow$) and final states with double circles.
   b. Identify formally the input alphabet Σ, the output alphabet Γ, the set of states Q, and the set of accepting states F.
2. **[50%]** Implement your FST in `pyformlang`.
   a. Build the transducer using `FST.add_transitions()`, `add_start_state()`, and `add_final_state()`. Complete the skeleton below, adding all missing transitions:

```python
from pyformlang.fst import FST

fst = FST()
fst.add_transitions([
    # ... complete all transitions here
])
fst.add_start_state('q0')
# fst.add_final_state('qx')

results = list(map(lambda x: "".join(x), list(fst.translate('musica'))))
print(results)
```

   b. Verify your implementation against the test case below. Recall: `translate()` returns a generator of output-symbol lists; use `list(map(lambda x: "".join(x), list(...)))` to collect all outputs as strings. Your result must contain **exactly** the five expected strings — no more, no less.

Input	Expected outputs (any order)	Got
`musica`	`['music', 'musique', 'musik', 'musiek', 'musikk']`	